

5. 低维数据可视化

5.1 一维标量数据可视化

一维标量数据可视化通常用二维坐标图或折线图可视化。本文通过1985年美国联邦调查局的男女犯罪类别统计数据进行可视化。首先通过大模型获取数据。

请给成1985年美国联邦调查局的男女犯罪类别统计数据

如果不能得到预期的数据，可以追问得到更多的数据，并整理成一个csv数据。 

[1985CrimeGender.csv](#)

5.1.1 数据转换

首先加载数据。仿照之前的数据加载和数据分列，完成以下代码。

Code block

```
1  <script>
2      // 启动渲染
3      main();
4      // 测试树渲染
5      async function main() {
6          try {
7              const data = await await d3.csv("data/1985CrimeGender.csv",
function (d) {
8                  return { crimeType:d.犯罪类型, male: d.男性, female: d.女性};
9              });
10             data.forEach(rec => {
11                 rec.genderDiffRatio = (rec.male-
rec.female)/(rec.male+rec.female);
12             });
13             data.sort((a, b) => b.genderDiffRatio - a.genderDiffRatio); //排序
14             console.log(data);
15         } catch (error) {
16             console.error("渲染失败:", error);
17             d3.select("#tree-container")
18                 .html(`<div style="color:red">加载失败: ${error.message}</div>`);
19         }
20     }
21 </script>
```

但得到的结果中，data的genderDiffRatio总是为NaN，将这段代码放到大语言模型中，给出两点纠正。

Code block

```
1 //第8行
2 return { crimeType:d.犯罪类型, male: +d.男性, female: +d.女性};
3 //第11行, rec.male被错误拼写为rec.maler
```



语法糖

简单数据生成，代码生成和代码矫正是大模型的基本功能，在时代大背景下要善加利用。

5.1.2 坐标轴变换

如果需要将数据进行可视化，需要进行两个核心坐标变换阶段。



原始数据 → [数据规范化] → 坐标系空间 → [比例尺映射] → 屏幕空间

数据空间 → 坐标系空间 的目标是将原始数据映射到数学上的坐标系（如笛卡尔坐标系、极坐标系等）。关键步骤：1) 选择坐标系类型：根据图表类型决定坐标系（直角坐标、极坐标、地理坐标等）；2) 数据规范化：将原始数据缩放/归一化到坐标系范围内（例如 $[0, 100]$ → 转换为坐标轴的 $[0, 10]$ 单位长度）；3) 映射到逻辑坐标：通过比例尺（Scale）将数据值映射到坐标系中的位置。D3.js 为这个变换提供了完整的函数支撑，包括线性尺度（用于定量数据，e.g. `d3.scaleLinear([10, 130], [0, 960]);`）、序数尺度（用于分类或序数数据，e.g. `d3.scaleOrdinal(["a", "b", "c"], ["red", "green", "blue"]);`）、带尺度（用于位置编码的分类或序数数据）等，详细内容参考[d3.scale](#)。

坐标系空间 → 屏幕空间 的目标是将数学坐标系中的点转换为屏幕/画布上的像素位置。关键步骤：1) 定义屏幕范围：确定画布的实际像素尺寸（如 `width=800px, height=600px`）；坐标比例尺转换：通过比例尺将逻辑坐标映射到像素坐标；处理方向与偏移：调整坐标系原点位置（如屏幕坐标系通常以左上角为 $(0,0)$ ）。这个映射在D3.js中会自动完成。

首先，为了保证代码的可复用性，设置配置文件scatterConfig.json，将独特性信息保存在配置文件中。

scatterConfig.json

```
1 {
2   "chartid" : "#chart",
3   "margin" : { "top": 40, "right": 100, "bottom": 60, "left": 200 },
4   "width" : 800,
5   "height" : 600,
```

```

6     "x_title" : "性别差比例 (男性比例 - 女性比例)",
7     "y_title" : "犯罪类型",
8     "index_col" : "crimeType",
9     "key_col" : "genderDiffRatio"
10  }

```

然后进行绘制。

Code block

```

1  async function renderScatter(data) {
2      cfg = await d3.json("utils/scatterConfig.json");
3      setupChart(cfg); // 设置图表尺寸和边距
4      setupScale(data, cfg); // 设置比例尺和坐标轴
5      const scatter = createScatter(data, cfg); // 绘制数据
6      tooltip(scatter);
7  }

```

第一部分是设置图表和边距。

Code block

```

1  function setupChart(cfg) {
2      // 创建SVG元素
3      cfg.svg = d3.select(cfg.chartid)
4          .append("svg")
5          .attr("width", cfg.width) // 设置屏幕坐标范围
6          .attr("height", cfg.height) // 设置屏幕坐标范围
7          .append("g")
8          .attr("transform", `translate(${cfg.margin.left},${cfg.margin.top})`);
9      // 设置绘画区
10     cfg.width -= (cfg.margin.left+cfg.margin.right);
11     cfg.height -= (cfg.margin.top+cfg.margin.bottom);
12 }

```

第二部分是设置比例尺和坐标轴。

Code block

```

1  function setupScale(data, cfg) {
2      // 创建比例尺
3      cfg.x = d3.scaleLinear() // 线性变换
4          .domain([d3.min(data, d => d[cfg.key_col]) - 0.1,
5                  d3.max(data, d => d[cfg.key_col]) + 0.1])
6          .range([0, cfg.width]);

```

```

7
8   cfg.y = d3.scaleBand() // 类别型映射
9       .domain(data.map(d => d[cfg.index_col]))
10      .range([0, cfg.height])
11      .padding(0.2);
12
13  // 添加x轴
14  cfg.svg.append("g")
15      .attr("transform", `translate(0,${cfg.height})`)
16      .call(d3.axisBottom(cfg.x).ticks(10))
17      .append("text")
18      .attr("class", "axis-label")
19      .attr("x", cfg.width / 2)
20      .attr("y", 40)
21      .attr("fill", "#000")
22      .text(cfg.x_title);
23
24  // 添加y轴
25  cfg.svg.append("g")
26      .call(d3.axisLeft(cfg.y))
27      .append("text")
28      .attr("class", "axis-label")
29      .attr("transform", "rotate(-90)")
30      .attr("y", -50)
31      .attr("x", -cfg.height / 2)
32      .attr("fill", "#000")
33      .text(cfg.y_title);
34  }

```

这里是绘制坐标轴时非常重要的一步，注意这里的生成的x和y是两个映射函数，形成从数据域domain到值域range的映射方式。d3.scale最重要的功能就是将数值映射到坐标系。特别注意这时并没有具体数据，直到根据具体数据绘制散点时，才会把数值转换为具体坐标。线性映射scaleLinear常用于数值类型，而scaleBand用于类别型或序数型数据的映射。D3.js中存在大量的映射函数，但这两个函数是最常用的。

? 思考

请完成下图的实验，当输入框中的内容发生变化时，显示的图形跟随发生变化。

Domain:

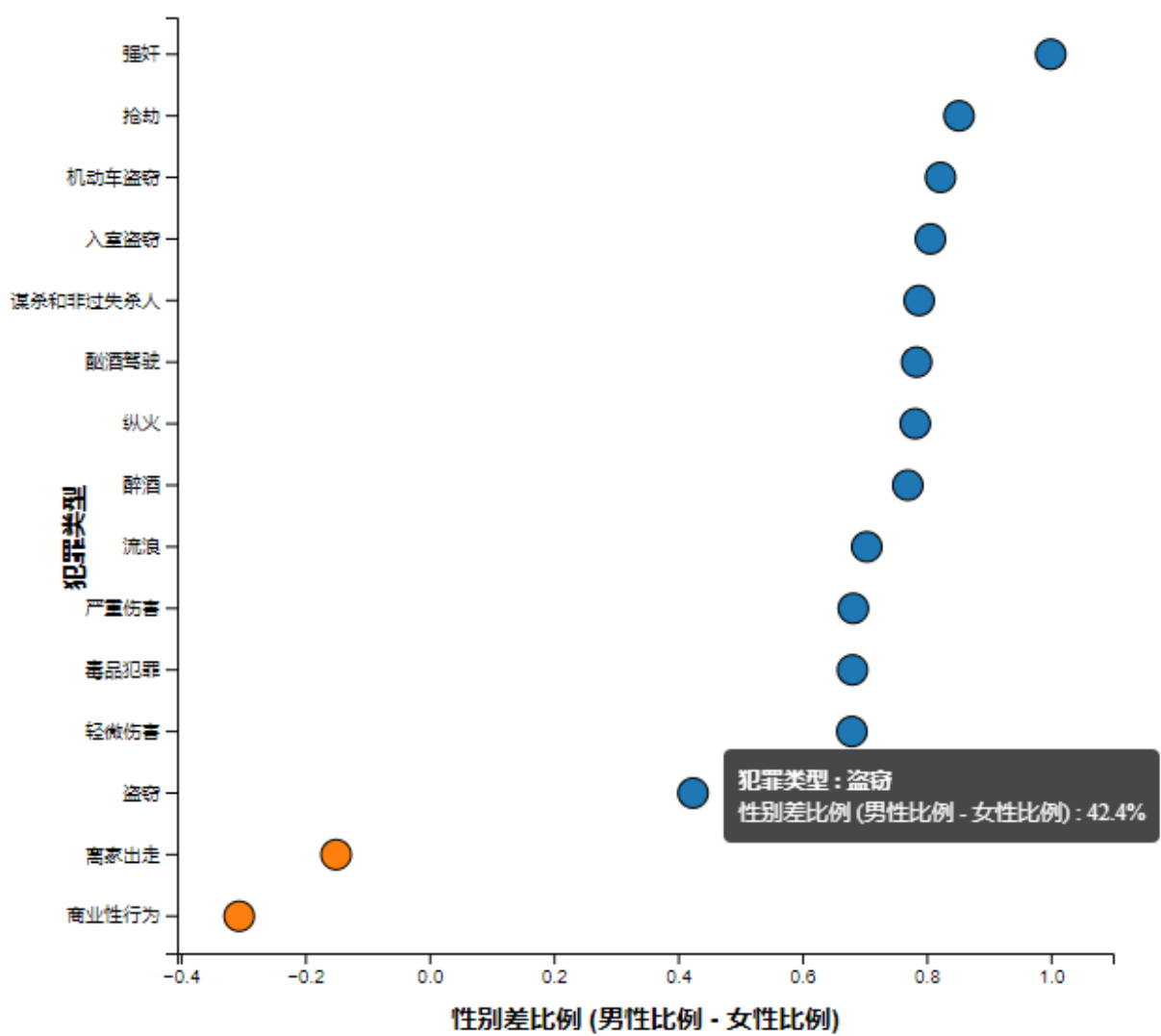


最后，将数据转换为散点，进行绘制。

Code block

```
1  function createScatter(data, cfg){
2    // 添加散点
3    const scatter = cfg.svg.selectAll(".dot")
4      .data(data)
5      .enter()
6      .append("circle")
7      .attr("class", "dot")
8      .attr("cx", d => cfg.x(d.genderDiffRatio))
9      .attr("cy", d => cfg.y(d.crimeType) + cfg.y.bandwidth() / 2)
10     .attr("r", 8)
11     .attr("fill", d => d.genderDiffRatio >= 0 ? "#1f77b4" : "#ff7f0e");
12     return scatter;
13   }
14
15   function tooltip(scatter) {
16     // 创建工具提示
17     const tooltip = d3.select("body").append("div")
18       .attr("class", "tooltip")
19       .style("opacity", 0);
20     // 添加工具提示
21     scatters.on("mouseover", function (event, d) {
22       tooltip.transition()
23         .duration(200)
24         .style("opacity", .9);
25       tooltip.html(`<strong>${cfg.y_title} : ${d[cfg.index_col]}</strong>
26
27         <br/>
28         ${cfg.x_title} : ${d[cfg.key_col] * 100}.toFixed(1)}%
29
30       <br/>`);
31       tooltip.style("left", (event.pageX + 10) + "px")
32         .style("top", (event.pageY - 28) + "px");
33     });
34     tooltip.on("mouseout", function () {
35       tooltip.transition()
36         .duration(500)
37         .style("opacity", 0);
38     });
39   }
```

1985年美国犯罪数据性别差异比例（从大到小排序）



5.1.3 曲线拟合

通常一维标量数据用离散的点和折线表示。在特定情况下，把离散的点拟合成曲线更能体现数据的规律和趋势。曲线拟合有多种方式，包括分段折线、多项式拟合、高斯拟合等。很多拟合方法在让曲线经过或靠近离散点的同时，也考虑到曲线本身的光滑性，因为光滑的曲线更容易找到趋势和特征。

Code block

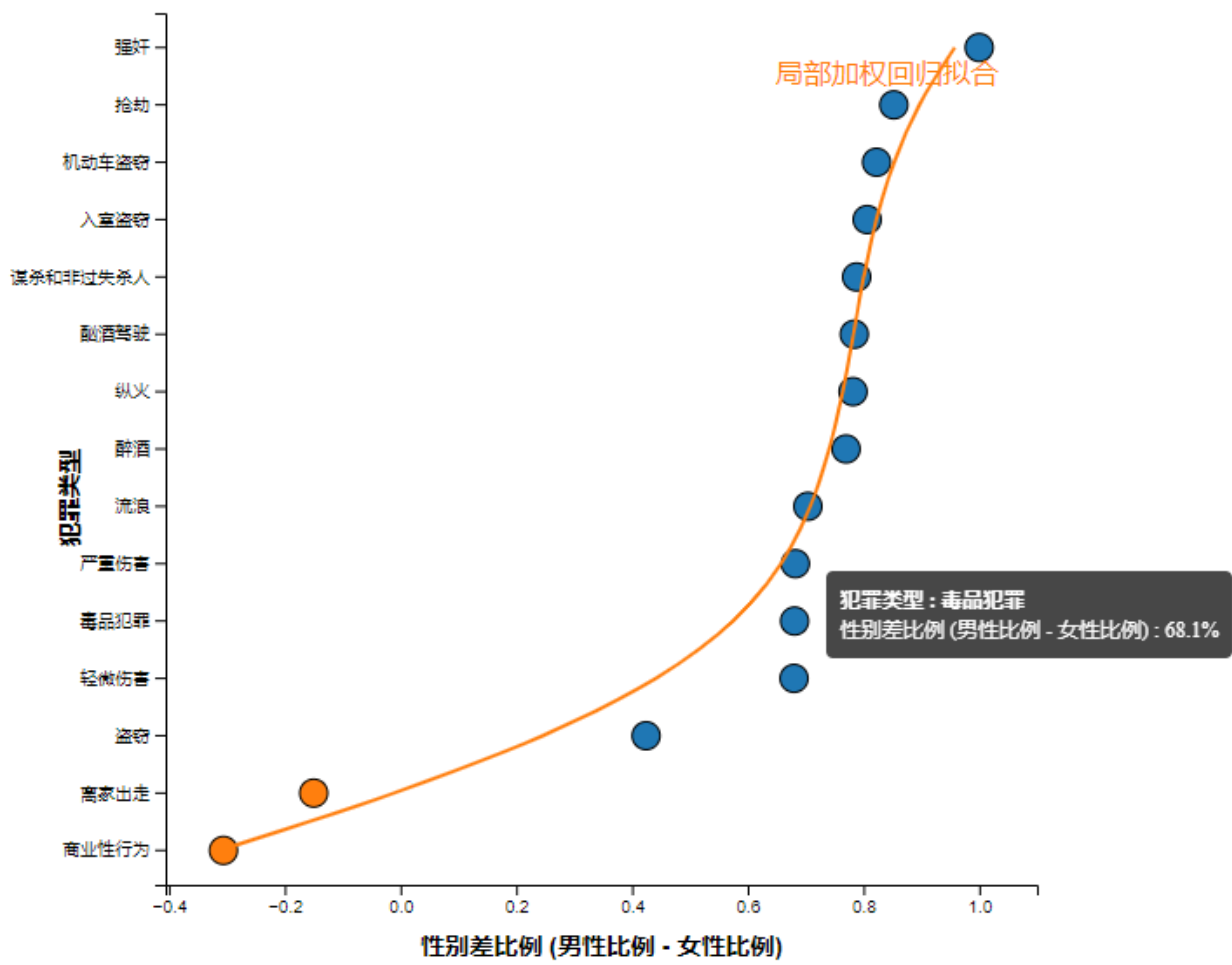
```
1 function curveRegression(data,cfg){
2   // 计算曲线拟合点 (使用局部加权回归)
3   const xValues = data.map((d, i) => i);
4   const yValues = data.map(d => d[cfg.key_col]);
5
6   // 使用二次多项式拟合
7   const Xs = interceptX(xValues, yValues);
8   const fitPoints = d3.zip(
9     Xs.map(x => cfg.x(x)),
```

```

10     data.map(d => cfg.y(d[cfg.index_col]) + cfg.y.bandwidth() / 2)
11 );
12
13 // 绘制曲线拟合线
14 cfg.svg.append("path")
15     .attr("class", "fit-line")
16     .attr("d", d3.line().curve(d3.curveCatmullRom.alpha(0.5))(fitPoints));
17
18 // 添加拟合线说明
19 cfg.svg.append("text")
20     .attr("x", cfg.width - 150)
21     .attr("y", 40)
22     .attr("fill", "#ff7f0e")
23     .text("局部加权回归拟合");
24 }
25
26 function interceptX(xValues, yValues) {
27     const Xs = [];
28     const bandwidth = 2; // 控制曲线平滑度的参数
29
30     for (let i = 0; i < xValues.length; i++) {
31         const weights = xValues.map((x) => Math.exp(-Math.pow((x - i) /
bandwidth, 2) / 2));
32         let sumX = 0, sumY = 0, sumXY = 0, sumX2 = 0, sumW = 0;
33
34         for (let j = 0; j < xValues.length; j++) {
35             const w = weights[j];
36             sumX += w * xValues[j];
37             sumY += w * yValues[j];
38             sumXY += w * xValues[j] * yValues[j];
39             sumX2 += w * xValues[j] * xValues[j];
40             sumW += w;
41         }
42
43         const slope = (sumW * sumXY - sumX * sumY) / (sumW * sumX2 - sumX *
sumX);
44         const intercept = (sumY - slope * sumX) / sumW;
45
46         // fitPoints.push([cfg.x(slope * i + intercept), cfg.y(data[i]
[cfg.index_col]) + cfg.y.bandwidth() / 2]);
47         Xs.push(slope * i + intercept);
48     }
49     return Xs;
50 }

```

1985年美国犯罪数据性别差异比例（从大到小排序）



5.1.4 平均倾斜度45度

从可视化的角度，让一位数据线的平均倾斜度接近45度，可获得最优的可视化效果。可以从以下角度理解。

- 黄金视角理论人类双眼的水平视野（约120°）远大于垂直视野（约60°），对45度倾斜的线条感知最自然。研究表明，45度角附近的线条在视网膜成像时，能均匀激活视锥细胞，避免因过度水平或垂直导致的视觉疲劳（参考Ware, C. *Information Visualization: Perception for Design*）。
- Gestalt连续性原则接近45度的线条符合格式塔心理学中的"良好连续性"法则，观察者能更流畅地追踪数据趋势，降低认知负荷。
- 眼动追踪实验：MIT可视化实验室研究发现，45度趋势线平均被观察时间比水平线少15%，但信息提取准确率高22%（*IEEE Vis 2018*）。
- 误差识别率测试：在45度斜率下，受试者对异常值的识别速度比极端斜率快1.8倍（Heer & Bostock, *ACM CHI 2010*）。

5.2 二维标量数据可视化

二维标量数据比一维更为常见，本节介绍平面型二维数据可视化的方法。

5.2.1 颜色映射法

颜色映射常用于二维标量数据可视化。使用颜色映射需要建立一张将数值转换为颜色的颜色映射表，再将二维空间中的标量值转换为颜色映射表的索引值并显示对应的颜色。第一步建立颜色映射表。颜色映射表中含有一个序列的颜色值。这种映射通常称为传输函数。第二步将标量数据转换为颜色的索引值。

在这个样例中，首先加载一个csv文件，构建一个由大模型生成的10*10二维浮点型矩阵。

Code block

```
1  async function main() {
2      try {
3          // 修正1: 使用正确的参数顺序和格式
4          const csvText = await d3.text("data/terrain10.csv");
5          const data = d3.csvParseRows(csvText, function (row) {
6              // 对每一行进行处理，将字符串转换为数值
7              return row.map(function (column) {
8                  return isNaN(column) ? column : +column;
9              });
10         }); // 直接返回原始行
11         console.log("加载成功:", data);
12         renderHeatmap(data, "#contour-map");
13     } catch (error) {
14         console.error("加载CSV文件失败:", error);
15     }
16 }
```

通常采用 `d3.csv` 加载csv数据，或加载全部文本，然后用 `d3.csvParse` 函数进行解析。但是在本节的样本数据中，仅仅是10行数据，每行有10个浮点数，并没有列标题。因此先用 `d3.text` 加载所有数据，然后用 `d3.csvParseRows` 进行解析。但是这个函数加载的所有数据的字符串类型，因此用其第二个参数进行数据类型转换。

接下来绘制热度图。

Code block

```
1  function renderHeatmap(gridData, chartid) {
2      // 1. 检查数据范围 (调试用)
3      const minValue = d3.min(gridData, row => d3.min(row));
4      const maxValue = d3.max(gridData, row => d3.max(row));
5      console.log("Data range:", minValue, maxValue);
6
7      // 2. 设置画布
8      const width = 600, height = 600;
9      const margin = { top: 40, right: 40, bottom: 40, left: 40 };
10     const innerWidth = width - margin.left - margin.right;
```

```

11     const innerHeight = height - margin.top - margin.bottom;
12
13     const svg = d3.select(chartid)
14         .append("svg")
15         .attr("width", width)
16         .attr("height", height)
17         .append("g")
18         .attr("transform", `translate(${margin.left}, ${margin.top})`);
19
20     // 3. 颜色映射
21     const valueExtent = [minValue, maxValue];
22     const colorScale = d3.scaleSequential(d3.interpolateViridis)
23         .domain(valueExtent);
24
25     // 4. 使用 gridData (二维数组) 直接绘制热力图
26     const cellWidth = innerWidth / gridData[0].length; // 根据网格列数计算单元格宽
    度
27     const cellHeight = innerHeight / gridData.length; // 根据网格行数计算单元格高
    度
28
29     // 5. 为每一行数据创建一个 <g> 分组，并设置垂直偏移 (y值)
30     const row = svg.selectAll(".row")
31         .data(gridData) // 绑定二维数组，每个元素是一行数据
32         .enter().append("g")
33         .attr("class", "row")
34         .attr("transform", (d, i) => `translate(0, ${i * cellHeight})`); // 按行
    索引定位
35
36     // 6. 在每个行分组中，为每个数据点 (列) 创建一个矩形
37     row.selectAll(".cell")
38         .data(d => d) // 绑定每一行内部的数据数组
39         .enter().append("rect")
40         .attr("class", "heatmap-cell")
41         .attr("x", (d, i) => i * cellWidth) // 按列索引定位
42         .attr("y", 0) // y定位已由父级<g>的transform完成
43         .attr("width", cellWidth)
44         .attr("height", cellHeight)
45         .attr("fill", d => colorScale(d)); // d 直接是当前数据点的数值
46 }

```

第1部分检查数据范围，为下一步的数据映射颜色做准备。第3部分用序数型scale构建数值到颜色的映射关系。其中 `d3.interpolateViridis` 是d3中预制的一个颜色映射表。将所有数值映射为颜色。第5部分根据二维数据创建所有行，每行是一个 `<g>` 元素。在SVG（可缩放矢量图形）中，`<g>` 是分组元素（Group Element），用于将多个图形元素组合成一个逻辑单元。组内所有元素继承 `<g>` 的公共属性（如 `fill`，`stroke`，`transform`），通过嵌套 `<g>` 实现复杂图形的层级

管理。一行数据和一个 `<g>` 元素绑定。然后在第6部分为每行创建多个矩形，用之前实现的 `colorScale` 传输函数，将不同的数值映射成不同的颜色。注意第6部分的 `y` 都是0，因为在第5部分 `<g>` 元素在纵向上已经做好偏移，这里的 `y=0` 是相对 `group` 的偏移。

5.2.2 等值线提取法

等值线是另一类二维标量数据的可视化方法，通常用来提取二维标量数据中的某个特征，展示和分析特征的空间分布规律。等值线应用广泛，例如，地图上的等高线、天气预报中的等压线和等温线等。等值线上所有点的数值相同，称为等值，等值线将二维空间划分为等值线的内部和外部两个区域。等值线表示是二维数据中的重要模式和特征的表达方法，如医学影像中的组织边界、大气数值数据中的低压区和降雨区的边缘等。

Marching squares是计算机图形学中一个用于矩阵网格点数据生成等值面的一个算法。其主要思想是将每个 2×2 网格，其中4个顶点网格点根据等值线阈值处理为0，1标量。1代表数值大于阈值，0代表数值小于阈值。这样处理之后有16种情况， 2×2 网格点根据左上角顺时针编号。

Look-up table contour lines

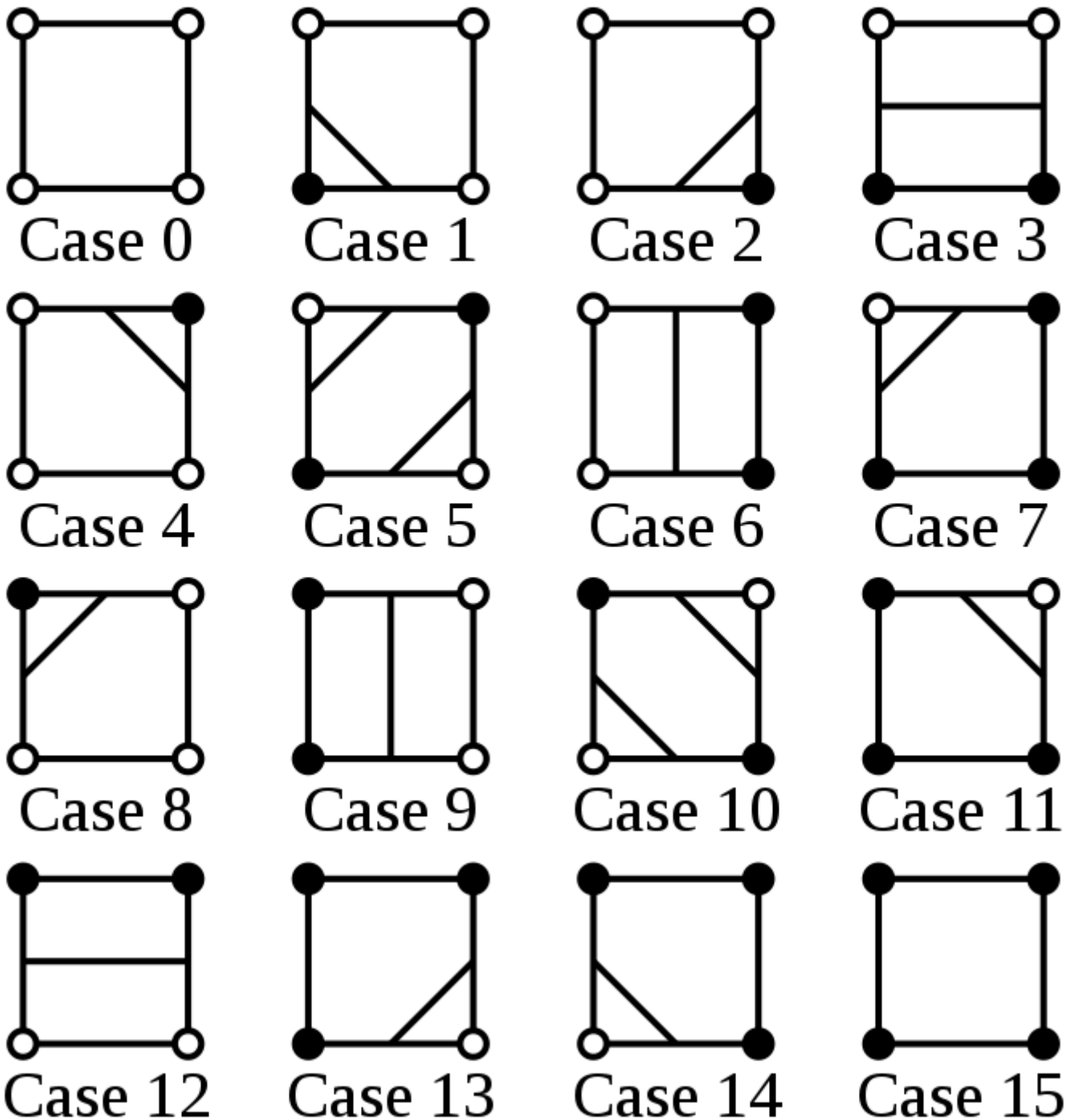


图5.x Marching Squares的16种情况

Code block

```
1 function marchingSquares(threshold) {  
2     const lines = [];  
3     const cellSizeX = config.canvasWidth / (config.gridSize - 1);  
4     const cellSizeY = config.canvasHeight / (config.gridSize - 1);  
5     console.log("计算等值线, 阈值:", threshold);  
6 }
```

```

7      d3.range(config.gridSize - 1).forEach(i => {
8          d3.range(config.gridSize - 1).forEach(j => {
9              // 获取四个顶点值
10             const v4 = [bl, br, tr, tl] = [
11                 gridData[i + 1][j], gridData[i + 1][j + 1],
12                 gridData[i][j + 1], gridData[i][j] ];
13
14             // 计算状态码 (使用d3.sum简化)
15             let state = d3.sum([1,2,4,8].map((w,index) => (v4[index] >=
threshold) * w));
16
17             // 计算当前四边形的坐标
18             const x = j * cellSizeX;
19             const y = i * cellSizeY;
20
21             // 使用D3的线性比例尺进行插值
22             const interpolateX = d3.scaleLinear()
23                 .domain([0, 1])
24                 .range([x, x + cellSizeX]);
25             const interpolateY = d3.scaleLinear()
26                 .domain([0, 1])
27                 .range([y, y + cellSizeY]);
28
29             // 计算插值参数t
30             const getT = (v1, v2) => (threshold - v1) / (v2 - v1);
31
32             const left = [x, interpolateY(getT(tl, bl))];
33             const right = [x + cellSizeX, interpolateY(getT(tr, br))];
34             const top = [interpolateX(getT(tl, tr)), y];
35             const bottom = [interpolateX(getT(bl, br)), y + cellSizeY];
36
37             // 根据状态码确定等值线
38             code = [[], [[left, bottom]], [[bottom, right]], [[left, right]],
39                 [[right, top]], [[left, top], [right, bottom]],
40                 [[bottom, top]], [[top, left]], [], [], [[left, bottom], [right,
top]],
41                 [], [], [], [], []];
42             [1, 2, 3, 4, 6, 7].forEach(d => code[15 - d] = code[d]);
43             lines.push(...code[state]);
44         })
45     });
46
47     console.log("生成等值线段数量:", lines.length);
48     return lines;
49 }

```

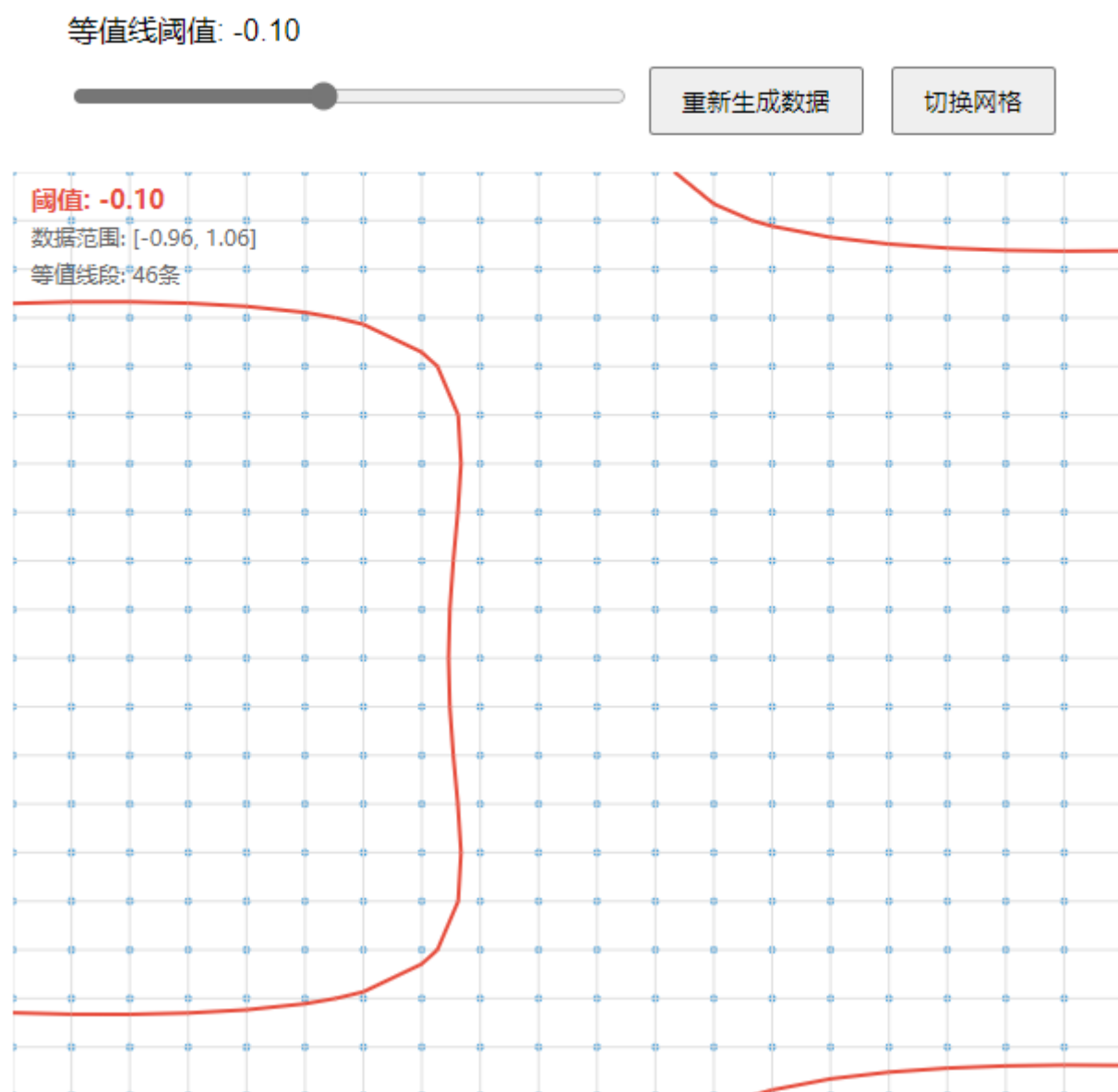
第15行根据四个顶点与阈值的大小比较情况，求和取得图5.x的16种情况的索引编码。仔细观察图5.x从左上角顶点开始按顺时针的黑白情况，把其作为二进制就可以得到对应情况的索引编码。第22-27行利用线性比例尺，将归一化的数值映射为逻辑坐标。然后使用 `getT` 将实际数值进行归一化后的插值。然后在第32-35行给出了在每个方形四个边上的插值点。第38行给出了8种情况的等值线绘制，另外6种因为对称关系，有第42行给出。因为第5和10两种情况要绘制两条线，因此在设置时每种情况都设置为二维数组，方便第43行用push+展开运算符的统一形式，添加到等值线段的集合中。

对于生成好的线段集合，采用以下代码进行绘制。

Code block

```
1  function drawContours(svg, contourLines) {
2      console.log("绘制等值线，线段数量:", contourLines.length);
3      // 绘制等值线 (使用d3.line)
4      svg.selectAll(".contour-line")
5          .data(contourLines.filter(line => line && line.length === 2))
6          .join("path")
7          .attr("class", "contour-line")
8          .attr("d", d3.line(d => d[0], d => d[1]))
9          .attr("data-index", (d, i) => i);
10 }
```

移动四边形法等值线提取



完整代码下载链接 [contourd3.html](#)

5.2.3 标记法

标记是离散的可视化元素，可采用标记的颜色、大小和形状等直接进行可视表达，而不需要对数据进行插值等操作。如果标记布局稀疏，还可以设计背景图形显示其他数据，并将标记和背景叠加在一个场景中，达到多变量可视化的目的。以下代码在颜色映射法的基础上，去除填充色添加如下代码。核心思想就是将值映射为填充标记的边长。

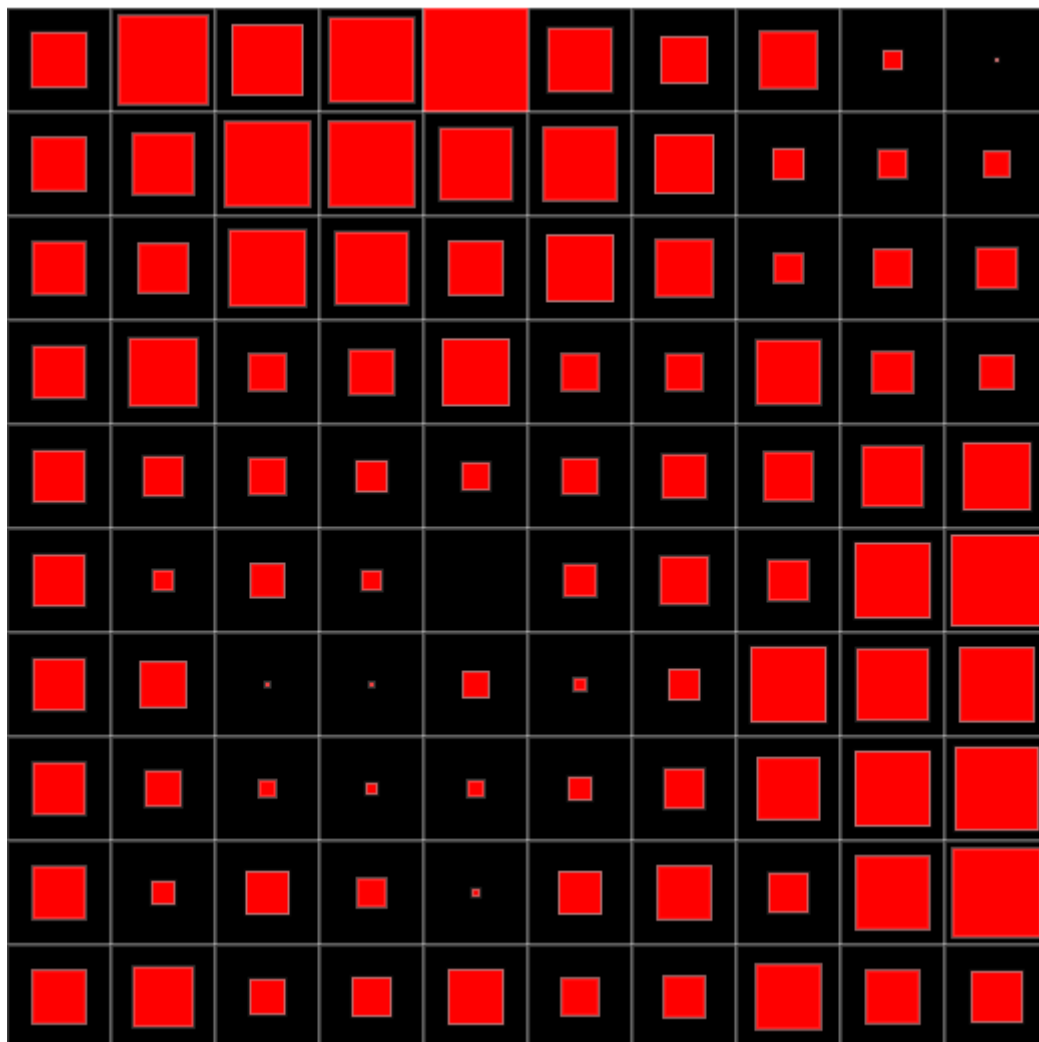
Code block

```
1  const valScale = d3.scaleSequential([0, 1])
2    .domain(valueExtent);
3  row.selectAll(".mark")
4    .data(d => d) // 绑定每一行内部的数据数组
5    .enter().append("rect")
```

```

6      .attr("class", "heatmap-cell")
7      .attr("x", (d, i) => -valScale(d) * cellWidth / 2)
8      .attr("y", (d, i) => -valScale(d) * cellHeight / 2)
9      .attr("width", (d, i) => valScale(d) * cellWidth)
10     .attr("height", (d, i) => valScale(d) * cellHeight)
11     .attr("transform", (d, i) => `translate(${(i + 0.5) * cellWidth}, ${0.5 *
cellHeight})`)
12     .attr("fill", "red");

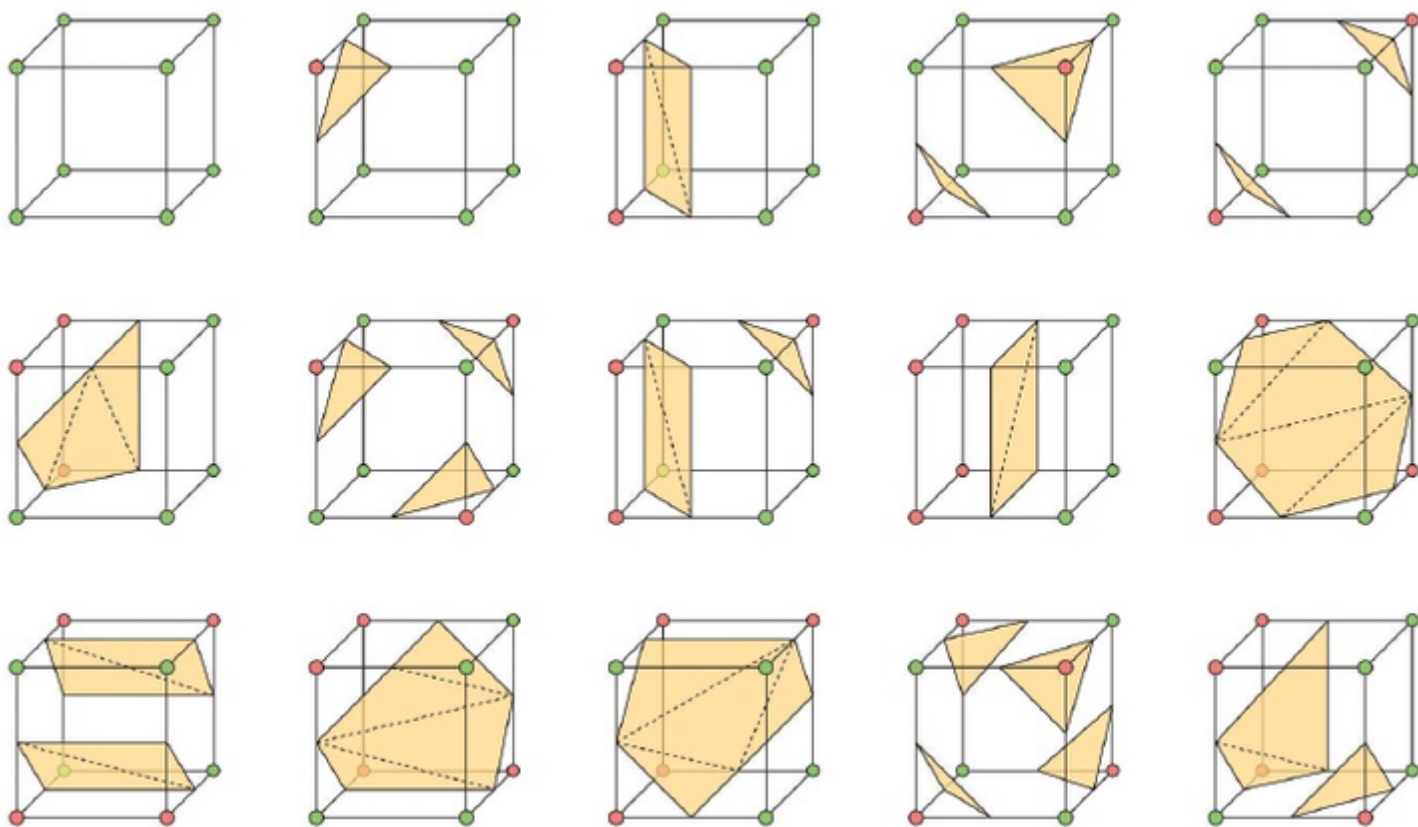
```



5.3 三维标量数据可视化

5.3.1 等值面提取法

等值面提取法在三维中，与二维中的等值线提取法类似，在此就不一一赘述了。Marching cubes 具有如下15种情况。



5.3.2 直接体绘制

直接体绘制不提取几何表示，直接呈现三维空间标量数据中的有用信息。因为D3.js主要针对二维可视化表达，因此在此不进行详细描述。